UTEC
Universidad Tecnológica

# Colas (Queue)

Una cola (Queue) es una estructura de datos lineal que sigue el principio FIFO (First In, First Out), lo que significa que el primer elemento en ser añadido es el primero en ser eliminado. Es similar a una fila de personas esperando su turno.

## Características principales:

- FIFO: El primer elemento en entrar es el primero en salir.
- Acceso Limitado: Solo se pueden realizar operaciones en los extremos de la cola: añadir elementos al final y eliminar elementos del principio.
- Implementación: Puede implementarse utilizando varias estructuras como arrays, listas enlazadas, o clases específicas en Java como `LinkedList` y `PriorityQueue`.

## Operaciones básicas:

1. Enqueue (offer): Añade un elemento al final de la cola.
2. Dequeue (poll): Elimina y devuelve el elemento al principio de la cola.
3. Peek: Devuelve el elemento al principio de la cola sin eliminarlo.
4. IsEmpty: Verifica si la cola está vacía.
5. Size: Devuelve el número de elementos en la cola.

## Uso recomendado:

- Gestión de Tareas: Ideal para manejar tareas en orden de llegada.
- Sistemas de Mensajes: Útil en la gestión de mensajes donde se necesita procesar los mensajes en el orden en que llegaron.
- Control de Flujos: Empleada en sistemas de control de flujo de datos y en algoritmos de búsqueda en anchura (BFS) en grafos.

## Implementación con `LinkedList` en Java:

```java
import java.util.LinkedList;
import java.util.Queue;

public class EjemploCola {
    public static void main(String[] args) {
        Queue<Integer> cola = new LinkedList<>();
```

UTEC
Universidad Tecnológica

```
        // Añadir elementos a la cola
        cola.offer(1); // [1]
        cola.offer(2); // [1, 2]
        cola.offer(3); // [1, 2, 3]

        // Mostrar la cola actual
        System.out.println("Contenido de la cola: " + cola);

        // Obtener y eliminar el primer elemento de la cola (Dequeue)
        int primerElemento = cola.poll(); // 1, cola ahora es [2, 3]
        System.out.println("Primer elemento eliminado de la cola: " +
primerElemento);

        // Mostrar la cola después de eliminar el primer elemento
        System.out.println("Contenido de la cola después de eliminar: "
+ cola);

        // Obtener el primer elemento sin eliminarlo (Peek)
        int primerElementoSinEliminar = cola.peek(); // 2
        System.out.println("Primer elemento de la cola (sin eliminar): "
+ primerElementoSinEliminar);

        // Mostrar la cola después de ver el primer elemento
        System.out.println("Contenido de la cola después de ver: " +
cola);

        // Verificar si la cola está vacía
        boolean estaVacia = cola.isEmpty(); // false
        System.out.println("¿La cola está vacía?: " + estaVacia);

        // Obtener el tamaño de la cola
        int tamaño = cola.size(); // 2
        System.out.println("Tamaño de la cola: " + tamaño);
    }
}
```

## Implementación con `PriorityQueue` en Java:

UTEC
Universidad Tecnológica

```java
import java.util.PriorityQueue;
import java.util.Queue;

public class EjemploColaPrioridad {
    public static void main(String[] args) {
        Queue<Integer> colaPrioridad = new PriorityQueue<>();

        // Añadir elementos a la cola
        colaPrioridad.offer(3); // [3]
        colaPrioridad.offer(1); // [1, 3]
        colaPrioridad.offer(2); // [1, 3, 2]

        // Mostrar la cola actual
        System.out.println("Contenido de la cola de prioridad: " +
colaPrioridad);

        // Obtener y eliminar el primer elemento de la cola de prioridad
(Dequeue)
        int primerElemento = colaPrioridad.poll(); // 1, cola ahora es
[2, 3]
        System.out.println("Primer elemento eliminado de la cola de
prioridad: " + primerElemento);

        // Mostrar la cola después de eliminar el primer elemento
        System.out.println("Contenido de la cola de prioridad después de
eliminar: " + colaPrioridad);

        // Obtener el primer elemento sin eliminarlo (Peek)
        int primerElementoSinEliminar = colaPrioridad.peek(); // 2
        System.out.println("Primer elemento de la cola de prioridad (sin
eliminar): " + primerElementoSinEliminar);

        // Mostrar la cola después de ver el primer elemento
        System.out.println("Contenido de la cola de prioridad después de
ver: " + colaPrioridad);

        // Verificar si la cola de prioridad está vacía
        boolean estaVacia = colaPrioridad.isEmpty(); // false
        System.out.println("¿La cola de prioridad está vacía?: " +
estaVacia);

        // Obtener el tamaño de la cola de prioridad
```

UTEC
Universidad Tecnológica

```
        int tamaño = colaPrioridad.size(); // 2
        System.out.println("Tamaño de la cola de prioridad: " + tamaño);
    }
}
```

## Consideraciones adicionales:

- `LinkedList` es una implementación flexible y común para colas en Java, adecuada para la mayoría de los usos generales.
- `PriorityQueue` es útil cuando los elementos deben ser procesados según una prioridad específica, no necesariamente en el orden en que fueron añadidos.
- Ambas clases (`LinkedList` y `PriorityQueue`) implementan la interfaz `Queue`, lo que proporciona una API consistente para el manejo de colas.